

Eventia

Resumen del proyecto — Épica 1 (Usuarios, Roles y Seguridad) y Épica 2 (Eventos e Inscripciones)

Grupo 1 - Ingeniería de Software II

Última actualización: 5 de septiembre de 2026

1. Enunciado del proyecto

La organización de eventos académicos, profesionales o institucionales requiere gestionar participantes, conferencistas, actividades y agenda del evento. El sistema busca centralizar estas tareas para mejorar el control de inscripciones, la planificación y el registro de asistencia.

El sistema deberá permitir:

- Crear y administrar eventos.
- Gestionar participantes y conferencistas.
- Organizar la agenda de actividades.
- Registrar la asistencia a sesiones o actividades.
- Administrar tipos de eventos como congresos, seminarios, talleres y jornadas.
- Gestionar inscripciones, cupos y listas de participantes.
- Generar certificados o registros de participación.
- Configurar tipos de eventos, categorías, roles, políticas de inscripción, límites de cupo y formatos de agenda.

2. Modelo entidad-relación

El modelo presentado contiene las entidades principales del dominio:

- `USUARIO` , `ROL_SISTEMA` y `USUARIO_ROL` para usuarios y roles.
- `TIPO_EVENTO` y `EVENTO` para la administración de eventos.
- `PARTICIPANTE` y `ROL_PARTICIPANTE` para las personas inscriptas.
- `ACTIVIDAD` , `CATEGORIA_ACTIVIDAD` , `CONFERENCISTA` y `ACTIVIDAD_CONFERENCISTA` para la agenda.
- `POLITICA_INSCRIPCION` e `INSCRIPCION` para las reglas de registro.
- `ASISTENCIA` y `CERTIFICADO` para participación y acreditación.

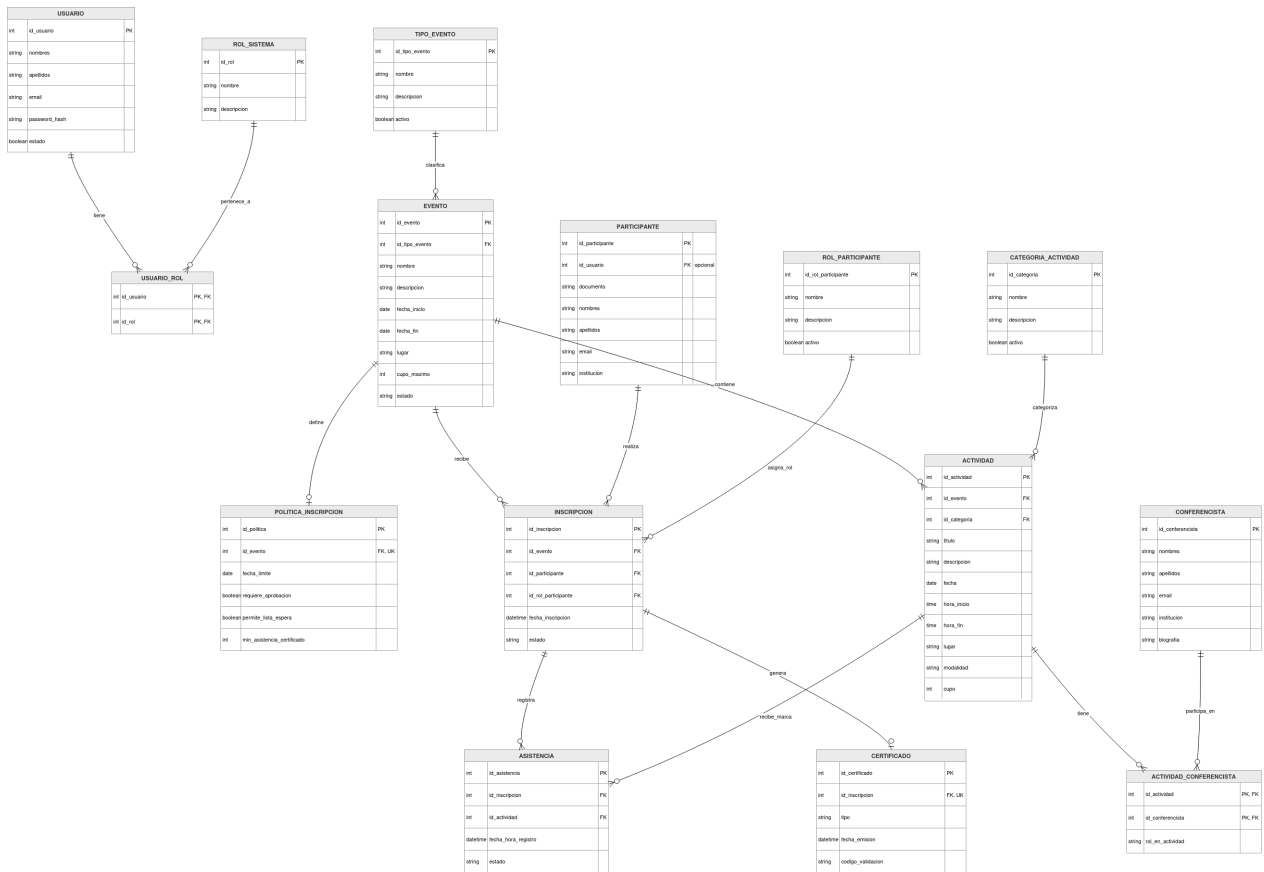


Diagrama entidad-relación utilizado como referencia del proyecto.

Con la Épica 2 (sección 4), **TIPO_EVENTO**, **EVENTO**, **POLITICA_INSCRIPCION**, **PARTICIPANTE** e **INSCRIPCION** dejaron de ser solo diseño: ya están modeladas en el ORM del backend (`app/models.py`) y respaldan los endpoints en producción.

3. Épica 1 — Gestión de Usuarios, Roles y Seguridad

El módulo contempla la administración de usuarios, la asignación de roles del sistema y el control de acceso a las funcionalidades. Las entidades centrales son:

- **USUARIO** : identidad, correo, estado y hash de contraseña.
- **ROL_SISTEMA** : roles disponibles y su estado.
- **USUARIO_ROL** : relación muchos a muchos entre usuarios y roles.

Como funcionalidades complementarias se identificaron el alta, edición y desactivación de usuarios, el inicio y cierre de sesión, la recuperación de contraseña y la autorización por roles.

3.1 HU01 — Autenticación de usuarios

Descripción: Como usuario del sistema, quiero iniciar sesión con mi correo y contraseña para acceder a las funcionalidades correspondientes a mi perfil.

La implementación utiliza el endpoint `POST /api/v1/auth/login`, valida las credenciales contra el hash seguro de contraseña y genera un token JWT.

Criterios de aceptación:

- Dadas credenciales válidas, el sistema compara la contraseña ingresada con `password_hash` mediante Argon2, verifica que el usuario esté activo y otorga acceso con un token de sesión JWT.
- Dadas credenciales incorrectas, el sistema responde con `401 Credenciales inválidas` sin especificar si falló el correo o la contraseña.

3.2 Middleware y tokens de sesión

Configurar la validación de tokens JWT para proteger los endpoints de la API.

- Generar un JWT con el identificador del usuario y sus roles.
- Definir una fecha de expiración.
- Leer el encabezado `Authorization: Bearer TOKEN`.
- Rechazar tokens ausentes, inválidos o vencidos.
- Permitir autorización basada en roles.

3.3 HU02 — Parametrización de roles de sistema y participantes

Como administrador, se pueden crear y administrar roles de usuario y tipos de participantes para controlar la seguridad y clasificar a los asistentes a un evento.

- Los roles de sistema y de participantes se administran como catálogos parametrizables.
- Los nombres son únicos, sin distinguir mayúsculas de minúsculas.
- Las operaciones administrativas requieren un JWT con rol `administrador`.
- La eliminación de un rol de participante se bloquea con conflicto `409` si existen inscripciones activas asociadas (incluyendo, desde la Épica 2, inscripciones en estado `lista_de_espera`).
- La eliminación de un rol de sistema se bloquea si está asignado a usuarios.

```
GET    /api/v1/roles/sistema
POST   /api/v1/roles/sistema
PUT    /api/v1/roles/sistema/{role_id}
DELETE /api/v1/roles/sistema/{role_id}

GET    /api/v1/roles/participantes
POST   /api/v1/roles/participantes
PUT    /api/v1/roles/participantes/{role_id}
DELETE /api/v1/roles/participantes/{role_id}
```

La API conserva PATCH como alias de actualización por compatibilidad.

4. Épica 2 — Gestión de Eventos e Inscripciones

Cubre las historias **HU03 (Creación y Parametrización de Eventos)** y **HU04 (Inscripción de Participante a un Evento)**.

Tarea	Descripción	Estado
3.2	POST /api/v1/eventos con validación de fechas ($\text{fin} \geq \text{inicio}$) y cupo positivo	Hecho
3.3	GET /api/v1/eventos (con filtros) y PUT /api/v1/eventos/{id}	Hecho
4.2	POST /api/v1/inscripciones con lógica transaccional (duplicado, cupo, estado)	Hecho
4.3	Derivación automática a "Lista de Espera" según la política del evento	Hecho

Las tareas de base de datos **3.1** y **4.1** (diseño formal de tablas `TIPO_EVENTO`, `EVENTO`, `POLITICA_INSCRIPCION`, `PARTICIPANTE`) no estaban asignadas a esta entrega, pero fue necesario modelarlas a nivel de ORM porque los endpoints de negocio dependen de ellas (ver sección 8, Notas y pendientes).

4.1 HU03 — Creación y Parametrización de Eventos

Descripción: Como Organizador, quiero crear un nuevo evento indicando su tipo, rango de fechas, lugar y cupo máximo para publicarlo y habilitar inscripciones.

Criterios de aceptación:

- Dado un formulario de alta de evento, cuando el organizador completa los campos obligatorios (tipo de evento, fechas válidas, cupo > 0), entonces el evento se guarda en estado "Borrador" o "Publicado".
- Dado un evento con fecha de fin anterior a la fecha de inicio, cuando se intenta guardar, entonces el sistema muestra un mensaje de error de validación.

4.2 HU04 — Inscripción de Participante a un Evento

Descripción: Como Participante, quiero inscribirme a un evento disponible para asegurar mi cupo y recibir mi comprobante.

Criterios de aceptación:

- Dado un evento con cupo disponible, cuando un participante envía su formulario de inscripción, entonces el sistema crea el registro con estado "Confirmada" y reduce el cupo disponible.
- Dado que un participante ya posee una inscripción activa en el mismo evento, cuando intenta inscribirse nuevamente, entonces el sistema bloquea la operación indicando que ya está registrado.

- Dado un evento sin cupos pero con la política "permite lista de espera" activa, cuando ingresa una nueva inscripción, entonces se registra en estado "Lista de Espera".

4.3 Modelo de datos agregado

- **TipoEvento**: catálogo parametrizable de tipos de evento (congreso, seminario, taller, jornada).
- **Evento**: tipo, nombre, descripción, fecha de inicio/fin, lugar, cupo máximo y estado (borrador / publicado).
- **PoliticaInscripcion**: relación 1 a 1 con **Evento**; define `permite_lista_espera`, `requiere_aprobacion`, `fecha_limite` y `min_asistencia_certificado` (con valores por defecto si no se especifican).
- **Participante**: datos de contacto de la persona que se inscribe (documento, nombres, apellidos, email único, institución). No requiere una cuenta de usuario del sistema.
- **Inscripcion** (extendida): ahora incluye `id_evento` e `id_participante`, con una restricción `UNIQUE (id_evento, id_participante)` que impide que un mismo participante quede inscripto dos veces al mismo evento.

4.4 Endpoints

app/api/eventos.py

POST /api/v1/eventos	Crea un evento (requiere rol "organizador")
GET /api/v1/eventos	Lista eventos con filtros (público)
PUT /api/v1/eventos/{evento_id}	Actualiza un evento (requiere rol "organizador")

- **Validaciones al crear/actualizar**: el `id_tipo_evento` debe existir (404 si no), `fecha_fin` no puede ser anterior a `fecha_inicio` (422), `cupos_maximo` debe ser mayor a 0 (validado por el schema).
- **Filtros del listado**: `id_tipo_evento`, `estado`, `q` (búsqueda por nombre), `fecha_desde` y `fecha_hasta` (eventos cuyo rango de fechas se solapa con el filtro).
- La política de inscripción se puede enviar anidada en el body (`politica`); si no se envía, se crea con valores por defecto (`permite_lista_espera=false`, `min_asistencia_certificado=75`).

app/api/inscripciones.py

POST /api/v1/inscripciones	Inscribe a un participante a un evento (público)
----------------------------	--

Lógica transaccional del endpoint:

1. Busca el evento (con bloqueo de fila `SELECT ... FOR UPDATE` para evitar condiciones de carrera al validar el cupo) y el rol de participante; responde `404` si no existen.

2. Busca un `Participante` existente por email (sin distinguir mayúsculas) o crea uno nuevo con los datos recibidos.
3. Si ya existe una inscripción de ese participante a ese evento, responde `409 Conflict`.
4. Cuenta las inscripciones con estado `confirmada` para el evento:
 - Si hay cupo disponible → estado `confirmada`.
 - Si no hay cupo pero `politica.permite_lista_espera` es verdadero → estado `lista_de_espera`.
 - Si no hay cupo y no se permite lista de espera → `409 Conflict`.
5. Persiste la inscripción con `fecha_inscripcion` (UTC) y la devuelve.

5. Implementación realizada

Se preparó un backend REST con Python, FastAPI, SQLAlchemy y PostgreSQL:

```
backend/
├── app/
│   ├── api/
│   │   ├── auth.py           # POST /api/v1/auth/login
│   │   ├── roles.py          # Parametrización de roles
│   │   ├── users.py          # GET /api/v1/users/me
│   │   ├── eventos.py        # Épica 2 (HU03): eventos
│   │   └── inscripciones.py   # Épica 2 (HU04): inscripciones
│   ├── core/
│   │   ├── config.py         # Variables de entorno
│   │   └── security.py       # Argon2 y JWT
│   ├── db.py                 # Conexión SQLAlchemy
│   ├── models.py             # Usuario, roles, evento, inscripción, etc.
│   ├── schemas.py            # Solicitudes y respuestas
│   └── main.py                # Aplicación FastAPI
├── .env.example
└── requirements.txt
```

El login devuelve el token, su tipo, el tiempo de expiración y los datos públicos del usuario.

Nunca devuelve `password_hash`. La parametrización usa `ROL_SISTEMA`, `ROL_PARTICIPANTE`, `USUARIO_ROL` e `INSCRIPCION`.

Con la Épica 2, el backend además crea y publica eventos, y gestiona su ciclo de inscripción completo (confirmación directa o lista de espera según cupo y política).

6. Ejecución local

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

```
cp .env.example .env
uvicorn app.main:app --reload
```

Configurar en `.env` la conexión a PostgreSQL:

```
DATABASE_URL=postgresql+psycopg://usuario:password@localhost:5432/eventia
JWT_SECRET_KEY=una-clave-secreta-de-al-menos-32-caracteres
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=60
```

La documentación interactiva está disponible en `http://127.0.0.1:8000/docs`. También existe el endpoint `GET /health` para comprobar que el servidor está activo.

7. Validaciones realizadas

- Compilación de todos los módulos Python mediante `compileall`.
- Registro correcto de las rutas de login, roles, eventos e inscripciones en OpenAPI.
- Hash y verificación de contraseñas mediante Argon2.
- Generación de JWT con roles y expiración.
- Suite completa de `pytest`: **19 tests pasan**.
 - `tests/test_auth.py`: autenticación válida, contraseña incorrecta y correo inexistente.
 - `tests/test_role_integrity.py`: bloqueo de borrado de roles en uso (incluye estado `lista_de_espera`).
 - `tests/test_eventos.py` (nuevo): creación con valores por defecto, rechazo de fechas inválidas, rechazo de tipo de evento inexistente, filtros de listado, actualización parcial y de política.
 - `tests/test_inscripciones.py` (nuevo): inscripción confirmada con cupo disponible, rechazo de duplicados, paso a lista de espera al agotar cupo, rechazo sin cupo ni lista de espera, reutilización de participante existente por email.

La prueba contra una base PostgreSQL real requiere configurar el servidor y crear las tablas del modelo.

8. Notas y pendientes

sql/ desactualizado respecto al ORM: los scripts de migración en `sql/` usan nombres de columna distintos a los del modelo real (`correo/contrasena_hash` vs. `email/password_hash` en `usuario`, por ejemplo) y todavía no incluyen las tablas de la Épica 2 (`evento`, `tipo_evento`, `politica_inscripcion`, `participante`, ni la `inscripcion extendida`). Los tests corren contra un esquema generado directamente desde `app/models.py` (SQLite en memoria), que es la fuente de verdad actual del

backend. Falta sincronizar o generar los scripts SQL correspondientes (tareas 3.1 y 4.1) para que el despliegue en PostgreSQL coincida con el ORM.

- **Rol "organizador":** no viene seedeado en ningún script; un administrador debe crearlo mediante `POST /api/v1/roles/sistema`, igual que cualquier otro rol parametrizable del sistema.